

Problema del Recorregut de Dues Peces en un Tauler

Pràctica 2

Serafí Nebot Ginard
Universitat de les Illes Balears (UIB)
Grau en Enginyeria Informàtica
Algorismes Avançats, Curs 2025–2026

Resum—Aquesta memòria descriu el disseny i la implementació d'una aplicació Java que resol el *problema del recorregut de dues peces*: donat un tauler $N \times M$ i dues peces d'escacs, determinar si existeix un camí hamiltonià en el qual les dues peces s'alternen per cobrir totes les caselles exactament una vegada, sense que cap de les dues pugui capturar l'altra en cap moment. L'aplicació implementa el patró MVC amb comunicació basada en events semàntics, utilitza backtracking recursiu millorat amb l'heurística de Warnsdorff, i incorpora una interfície gràfica interactiva que permet configurar el tauler, visualitzar el procés de resolució en temps real i reproduir la solució pas a pas.

Termes d'índex—backtracking, heurística de Warnsdorff, camí hamiltonià, Model-Vista-Controlador, SwingWorker, peces d'escacs, Java Swing

1. Introducció

El problema del recorregut del cavall (*knight's tour*) és un problema clàssic de la teoria de grafs i dels algorismes combinatoris: donat un tauler d'escacs, cal trobar una seqüència de moviments d'un cavall tal que visiti cada casella exactament una vegada. La generalització d'aquest problema a dues peces que s'alternen els moviments, amb la restricció addicional que cap de les dues pot capturar l'altra en cap pas del recorregut, és notablement més complexa.

L'aplicació implementada permet seleccionar dos tipus de peces de manera independent (Cavall, Reina, Torre, Alfíl, Mussol o Serp) i trobar, mitjançant backtracking, un recorregut alternat que cobreixi totes les caselles d'un tauler de mida configurable (mínim 2×2 , sense límit superior pràctic).

2. Descripció de l'Aplicació

L'aplicació és una eina interactiva d'escriptori (Java Swing) que permet plantejar i resoldre el problema del recorregut hamiltonià de dues peces, visualitzar la solució pas a pas i inspeccionar les mètriques de la cerca. La interfície es divideix en tres zones: un panell de configuració a l'esquerra, el tauler al centre i un panell de controls i estadístiques a la part inferior.

2.1. Configuració

2.1.1. Mida del tauler. L'usuari especifica el nombre de files i columnes amb dos *spinners* (mínim 2, sense límit superior pràctic). Un botó *toggle* etiquetat *Quadrat*, situat sobre els dos *spinners* i actiu per defecte, manté les dues

dimensions iguals: qualsevol canvi en un *spinner* es propaga automàticament a l'altre. Quan el botó està inactiu, files i columnes es poden ajustar de manera independent per obtenir taulers no quadrats ($N \times M$). El botó canvia de color de fons (verd quan actiu) per indicar l'estat d'un cop d'ull.

2.1.2. Selecció de peces. Cada una de les dues peces (blanca i negra) es configura de manera independent mitjançant un `JComboBox` amb les sis opcions disponibles: Cavall, Reina, Torre, Alfíl, Mussol i Serp. Una icona en miniatura mostra la imatge de la peça seleccionada (o les inicials en un cercle si la imatge no està disponible).

2.1.3. Posició inicial. La posició inicial de cada peça es pot establir de dues maneres:

- Introduint directament els valors de fila i columna (en base 1) als *spinners* de posició.
- Activant el mode de col·locació amb el botó *Col·locar al tauler* (color de fons taronja quan actiu) i fent clic sobre la casella desitjada al tauler. El mode es desactiva automàticament després de col·locar la peça.

En tot moment, les dues peces es mostren sobre el tauler buit en la seva posició configurada com a previsualització.

2.2. Execució de la cerca

El panell de controls inferior ofereix tres botons d'execució:

- **Iniciar:** valida la configuració, construeix el tauler i llança la cerca en un fil de fons.
- **Aturar:** envia un senyal cooperatiu al solver per a que aturi la cerca en curs.
- **Reiniciar:** atura la cerca (si n'hi ha una en curs), buida el tauler i restaura l'estat inicial.

Durant la cerca, tots els controls de configuració es deshabiliten per evitar inconsistències. El panell d'estadístiques mostra en temps real el nombre d'iteracions, de retrocesos, la profunditat màxima assolida i el temps transcorregut. Un indicador d'estat codificat per colors informa sobre la fase actual: gris (*Esperant*), blau (*Cercant*), verd (*Solució trobada*), vermell (*No hi ha solució*) i groc-marró (*Aturat*). Els colors específics es detallen a la Taula 2.

2.3. Visualització de la solució

Si la cerca troba una solució, el tauler es mostra complet amb totes les caselles visitades. Cada casella visitada indica el número de pas a la cantonada superior esquerra, i les icones de cada peça es mostren a la seva posició final. El tauler inclou etiquetes de fila (a l'esquerra) i columna (a la part superior), que s'alineen exactament amb cada casella.

L'usuari pot navegar per la solució pas a pas:

- **<< / >>:** retrocedeix o avança un pas. La casella del pas actual es resalta en groc i es pinta la peça corresponent.
- **Reproduir / Pausar:** inicia o atura una reproducció animada automàtica.
- **Lliscador de velocitat:** controla el retard entre passos de l'animació (50–2000 ms).

3. Disseny i Implementació

3.1. Patró MVC i Comunicació per Events

El patró MVC separa l'aplicació en tres capes:

- 1) **Model:** encapsula el tauler, el tipus de peces, l'algorisme de cerca i les mètriques d'execució.
- 2) **Vista:** presenta el tauler gràficament, permet configurar la mida i les posicions inicials, i ofereix controls de navegació i reproducció.
- 3) **Controlador:** rep les ordres de la vista, gestiona el cicle de vida del fil de cerca (*SwingWorker*) i actualitza la vista amb els resultats.

La comunicació entre capes es fa exclusivament a través d'events i callbacks.

La cerca de backtracking pot trigar des de mil·lisegons fins a hores o inclús dies per a taulers grans. Executar-la al fil EDT (*Event Dispatch Thread*) bloquejaria la interfície, impedit qualsevol interacció de l'usuari durant la cerca. La solució estàndard és *SwingWorker* [2] [3], una classe abstracta que encapsula el patró de treball en segon pla: el mètode *doInBackground()* s'executa en un fil separat del, mentre que *done()* s'invoca automàticament al fil

EDT quan la tasca finalitza. El progrés intermedi s'envia al fil EDT amb *SwingUtilities.invokeLater()*, permetent actualitzar les estadístiques en temps real sense bloquejar ni la cerca ni la interfície.

3.2. Capa del Model

La capa del model consta de cinc elements amb responsabilitats clares.

3.2.1. MovementType. Enumeració amb dos valors: *STATIC* i *CONTINUOUS*. Controla el tipus de moviment d'una peça: les peces estàtiques apliquen cada moviment una sola vegada, mentre que les contínues el repeteixen fins arribar al límit del tauler.

3.2.2. PieceType (Enum). Defineix els sis tipus de peces disponibles. Cada constant porta:

- Nom en català i abreviatura.
- *MovementType*: *STATIC* (un sol salt) o *CONTINUOUS* (lliscament).
- Matriu de vectors de moviment *int[][]*. Per a peces estàtiques, cada fila és el desplaçament (*dr, dc*) directe. Per a peces contínues, cada fila és una direcció que es repeteix fins a sortir del tauler o topar una casella visitada.
- *getImagePath(boolean isWhite)*: ruta al recurs de la icona PNG de la peça blanca si *isWhite=true* o negra si *isWhite=false*.

La Taula 1 resumeix les sis peces implementades.

Taula 1. PECES IMPLEMENTADES AMB EL SEU TIPUS DE MOVIMENT I NOMBRE DE VECTORS.

Peça	Tipus mov.	# mov.	Abrev.
Cavall	Estàtic	8	C
Reina	Continu	8	Q
Torre	Continu	4	T
Alfil	Continu	4	A
Mussol	Estàtic	8	M
Serp	Estàtic	8	S

Les peces especials *Mussol* i *Serp* s'han definit específicament per a aquesta pràctica. El Mussol es mou amb una variant ampliada del moviment del cavall: salta en forma de L amb passos de magnitud 3 i 1 (vectors $(\pm 3, \pm 1)$ i $(\pm 1, \pm 3)$), en comptes dels passos 2-1 del cavall estàndard. La Serp combina dos tipus de salt: un pas en qualsevol dels quatre sentits ortogonals (vectors $(\pm 1, 0)$ i $(0, \pm 1)$) i dos passos en qualsevol de les quatre diagonals (vectors $(\pm 2, \pm 2)$), donant un total de 8 vectors de desplaçament. La resta de peces tenen els seus moviments clàssics d'un tauler d'escacs.

3.2.3. Board. La classe *Board* gestiona l'estat del tauler amb una matriu booleana *boolean[][]* de caselles visitades i un comptador *visitedCount*. Proporciona:

- `getValidMoves(pos, piece)`: retorna les caselles accessibles des de `pos` per `piece` que no han estat visitades. Per a peces contínues, recorr cada direcció cel·la per cel·la fins a sortir del tauler o trobar una casella visitada.
- `getValidMovesSorted(pos, piece)`: igual, però ordena els moviments per l'heurística de Warnsdorff (vegeu Secció 4.2).
- `canCapture(attackerPos, attacker, targetPos)`: retorna `true` si `attacker` des de `attackerPos` pot assolir `targetPos`. Per a peces estàtiques, comprova si `targetPos` és exactament una aplicació d'algun vector de moviment. Per a peces contínues, recorr cada direcció cel·la per cel·la fins al límit del tauler i comprova si `targetPos` cau en alguna d'aquestes línies.

3.2.4. SolverMetrics. Registra estadístiques de la cerca: nombre d'iteracions, de retrocesos, profunditat màxima assolida i temps transcorregut. Permet monitoritzar l'eficàcia de l'heurística i comparar el comportament per a taulers i peces de diferent dificultat.

3.2.5. Position. Registre immutable que representa una casella del tauler amb les seves coordenades de fila i columna. Es podria substituir un per un simple un array de `int` de dos elements però l'ús de `record` (registre) ens permet referenciar cada coordenada de forma més fàcil i afegir mètodes per a realitzar càlculs sobre la posició. Per exemple: `add(pos)` per a sumar una posició a una altre; útil per a calcular el destí d'un moviment a partir d'una posició inicial.

3.3. Capa de la Vista

3.3.1. MainFrame. `JFrame` principal que compon quatre sub-panells amb `BorderLayout`:

- `ConfigPanel` a l'oest (en un `JScrollPane`).
- `BoardPanel` al centre.
- `ControlPanel` al sud.
- `StatsPanel` al sud (a la dreta de `ControlPanel`).

`MainFrame` és una classe de presentació pura: no manté cap estat de domini ni de navegació. El controlador hi interactua exclusivament a través dels mètodes públics de delegació (`resetBoard`, `updateStats`, `setSolverState`, etc.), que operen amb dades, mai amb components `Swing` directament.

3.3.2. BoardPanel. Panell de dibuix del tauler. Utilitza un `GridLayout` per a distribuir les cel·les del tauler. Cada casella és un `CellPanel` intern que:

- Dibuixa el color de fons (clar/fosc, o verd per a caselles visitades).
- Mostra el número de pas a la cantonada superior esquerra.

- Mostra la icona de la peça a la casella actual de cada peça escalada a la mida de la cel·la.
- Ressalta en groc la casella del pas actual durant la reproducció pas a pas.

El mètode `doLayout` calcula la mida de cel·la màxima entera que cap a l'espai disponible de la següent forma:

$$\text{cellSize} = \min(\lfloor w/\text{cols} \rfloor, \lfloor h/\text{rows} \rfloor)$$

on w és l'amplada del tauler i h l'alçada, i dimensiona la graella a $\text{cols} \times \text{cellSize}$ per $\text{rows} \times \text{cellSize}$, garantint que cada casella sigui un quadrat perfecte fins i tot per a taulers no quadrats ($N \times M$). Les etiquetes de fila i columna s'alineen llegint les coordenades reals de cada `CellPanel` i convertint-les al sistema de coordenades del panell exterior amb `SwingUtilities.convertPoint`. Les caselles responen al clic a través de `mousePressed`, en lloc de `mouseClicked`, per garantir resposta immediata i fiable: `mouseClicked` només s'activa si la posició del ratolí és idèntica entre prémer i alliberar el botó, cosa que falla si l'usuari mou el cursor fins i tot un sol píxel durant el clic [6].

El mètode `updateBoard(path, currentStep, whiteType, blackType)` actualitza tots els `CellPanel` donada la llista de posicions de la solució i el pas a visualitzar. La paritat del pas (parell/senar) determina quina peça ocupa la casella corresponent (els passos parells corresponen a la peça blanca, els senars a la negra).

3.3.3. ConfigPanel. Panell de configuració que implementa `Scrollable` per garantir que els controls siguin accessibles fins i tot quan la finestra es redimensioni a una alçada menor que l'espai requerit pels controls.

Des del punt de vista tècnic, els *spinners* de posició mostren valors en base 1 ($1 \dots N$ i $1 \dots M$) per a l'usuari, però els mètodes *getter* resten 1 internament per retornar índexs en base 0 ($0 \dots N - 1$ i $0 \dots M - 1$) al controlador. Els botons *toggle* actualitzen el seu color de fons (verd per al bloqueig de mida, taronja per al mode de col·locació) mitjançant `setBackground()` per fer l'estat visible sense modificar el text.

El mètode `validateConfig()` comprova que les posicions estiguin dins dels límits del tauler i siguin distintes, mostrant diàlegs d'error en català si la validació falla. Tota comunicació cap al controlador es fa a través d'un `Runnable` `onConfigChanged`, registrat exclusivament per `SolverController` en construir l'aplicació. Per tant, el panell no coneix el controlador, només invoca el callback quan detecta canvis de configuració.

3.3.4. ControlPanel. Conté sis botons i un lliscador de velocitat organitzats amb `BorderLayout`: els botons al centre i el lliscador a la part inferior. Els botons s'agrupen visualment en dos blocs separats per un `JSeparator` vertical: execució (Iniciar, Aturar, Reiniciar) i navegació (pas anterior, pas següent, reproduir/pausar). Tots els botons utilitzen símbols Unicode (▶, ■, ◯, etc.) combinats amb text en català, i incorporen tooltips descriptius.

El lliscador de velocitat controla el retard de l'animació de reproducció de la solució trobada entre 50 i 2000 ms, amb un valor per defecte de 500 ms. Cal destacar que el lliscador està invertit (`setInverted(true)`): el costat dret correspon a valors baixos de retard (animació ràpida), seguint la convenció intuïtiva on «dreta = més ràpid». En lloc de mostrar els valors numèrics per defecte (50, 2000) als extrems del lliscador, s'utilitza el mètode `setLabelTable()` amb un `Hashtable<Integer, JLabel>` que associa posicions específiques del lliscador amb etiquetes de text personalitzades («Lent», «Ràpid»). Aquest és l'únic mecanisme que l'API de `JSlider` ofereix per personalitzar les etiquetes, dissenyat per permetre col·locar etiquetes a qualsevol posició, no només als extrems.

El panell exposa tres mètodes d'estat explícits (`setIdleState`, `setExecutionState`, `setSolutionState`) que gestionen l'habilitació/deshabilitació dels botons segons la fase actual: durant l'execució de la cerca, tots els botons excepte «Aturar» es deshabiliten per evitar inconsistències; quan hi ha solució disponible, els botons de navegació s'habiliten. El botó de reproducció canvia dinàmicament el seu text entre «▶ Reproduir» i «|| Pausar» segons l'estat intern `isPlaying`.

3.3.5. StatsPanel. Utilitza `GridBagLayout` per organitzar parells etiqueta-valor en una disposició vertical compacta. Cada valor es mostra en negreta per destacar-lo visualment respecte de l'etiqueta. El panell té un `TitledBorder` amb el text «Estadístiques».

Mostra cinc mètriques en temps real: temps transcorregut, nombre d'iteracions, de retrocesos, profunditat màxima i estat actual. El temps es formata automàticament en format MM:SS o HH:MM:SS segons la durada, mitjançant el mètode `formatTime()`. Els nombres grans (iteracions, retrocesos) s'imprimeixen amb separadors de milers locals utilitzant `String.format("%,d", number)` per millorar la llegibilitat.

L'actualització del temps en directe es gestiona amb un `javax.swing.Timer` que s'executa cada 1000 ms (un segon) i recalcula el temps transcorregut des de `timerStartMillis` fins al moment actual. Aquest temporitzador s'inicia quan comença la cerca i s'atura quan finalitza.

Els estats es diferencien amb colors definits com a constants RGB. La Taula 2 mostra els cinc estats possibles amb els seus colors associats.

Taula 2. ESTATS DEL SOLVER I COLORS ASSOCIATS.

Estat	Definició Java
Esperant	<code>Color.GRAY</code>
Cercant	<code>new Color(0, 100, 200)</code>
Solució trobada	<code>new Color(0, 150, 0)</code>
No hi ha solució	<code>new Color(200, 0, 0)</code>
Aturat	<code>new Color(180, 120, 0)</code>

3.3.6. PieceIcon. Component reutilitzable que estén `JPanel` i renderitza la icona d'una peça escalada. Té una

mida preferida de 48×48 píxels amb un *padding* intern de 4 píxels. El panell és transparent (`setOpaque(false)`) per integrar-se visualment amb qualsevol fons.

Les imatges es carreguen des del classpath mitjançant `ImageIO.read()` amb un `InputStream`. Per evitar operacions d'I/O repetides, s'utilitza un `HashMap<String, BufferedImage>` estàtic (compartit amb totes les instàncies de `PieceIcon`) com a cache de les imatges indexades per la seva ruta de recurs.

El mètode `paintComponent()` utilitza `Graphics2D` amb dos *rendering hints* [4], [5]: `KEY_ANTIALIASING` perquè les icones es mostren sobre fons de colors variats i les vores irregulars serien especialment visibles, i `KEY_INTERPOLATION_BILINEAR` perquè les imatges s'escalen dinàmicament al tamany de cada casella i la interpolació bilineal evita la pixelació. Com que totes les imatges són quadrades, es calcula la mida màxima del quadrat que cap dins del component (`Math.min(w, h)`) i es centra horitzontalment i verticalment.

Si la imatge no es pot carregar, es dibuixa un *fallback* de text: un cercle amb el nom abreujat de la peça centrada. Les peces blanques utilitzen un fons beix clar (RGB 240, 240, 220) amb text negre, mentre que les peces negres utilitzen un fons marró fosc (RGB 80, 60, 40) amb text blanc. El cercle té un contorn de 1.5 píxels.

3.4. Capa del Controlador

La capa del controlador consta de tres classes: `BacktrackingSolver`, `SolverTask` i `SolverController`. Tot i que `BacktrackingSolver` implementa l'algorisme de cerca, s'ha classificat al controlador perquè orquestra el procés de resolució (gestiona els torns, el senyal d'aturada i els callbacks de progrés) sense mantenir cap estat de domini propi; l'estat del tauler resideix a `Board`.

3.4.1. BacktrackingSolver. El nucli algorísmic. Abans d'invocar `search()`, el mètode `solve()` reinicia el tauler, la llista de solució i les mètriques; marca les dues caselles inicials com a visitades i les afegeix a la llista; i comprova que les peces no es capturen mútuament a l'inici — si és el cas, retorna `false` immediatament sense iniciar el backtracking. El Listing 1 mostra l'esquelet general de l'algorisme backtracking en pseudocodi.

Per a cada candidat `next` es fan dues comprovacions de captura independents: que la peça actual no pugui capturar l'altra des de `next`, i que l'altra peça no pugui capturar la peça actual a `next`. Quan un moviment és vàlid, la posició es marca com a visitada; si la branca no porta a solució, es desfà (`unvisit`). L'intercanvi de rols a la crida recursiva (`otherPos`, `otherPiece`, `next`, `curPiece`) implementa l'alternança de torns. La profunditat màxima és $N \cdot M - 2$.

El progrés s'envia al fil EDT cada `PROGRESS_UPDATE_INTERVAL` (1000) iteracions via un `Consumer<Integer>` configurat per `SolverTask`,

```

1 boolean search(curPos, curPiece, otherPos,
2   otherPiece) {
3   if (all cells visited) return true;
4   if (stopped) return false;
5
6   for (next : validMovesSorted(curPos, curPiece))
7   {
8     if (canCapture(next, otherPos) || canCapture
9       (otherPos, next))
10      continue;
11
12    visit(next);
13    if (search(otherPos, otherPiece, next,
14      curPiece))
15      return true;
16    unvisit(next); // aquí es produeix el
17      retrocediment real
18  }
19  return false;
20 }

```

Listing 1. Esquelet del backtracking recursiu.

desacoblant el solver de qualsevol dependència de la vista. L'aturada cooperativa es gestiona amb un indicador volatíle boolean `stopped`: el bucle comprova `stopped` a cada iteració, i el controlador el posa a `true` quan l'usuari clica *Aturar*.

3.4.2. SolverTask. Estén `SwingWorker<Boolean, Void>`. El mètode `doInBackground()` registra un *callback* de progrés al solver que usa `SwingUtilities.invokeLater` per actualitzar el `StatsPanel` cada `PROGRESS_UPDATE_INTERVAL` iteracions (1000 per defecte), i llavors executa `solver.solve()`. El mètode `done()` recupera el resultat i l'envia a un `BiConsumer<Boolean, SolverMetrics>` que el controlador ha registrat en construir la tasca. L'aturada voluntària es propaga cridant `solver.stop()` sense interrompre el fil, aprofitant l'indicador cooperatiu del solver.

3.4.3. SolverController. Implementa el rol de controlador en el patró MVC i és l'únic dipositari de l'estat de navegació de la solució: la llista de posicions del camí hamiltonià, el pas actual i els tipus de les dues peces.

Registre de listeners.. Tot el registre d'esdeveniments es concentra al mètode privat `wireListeners()`, invocat al constructor. Aquesta decisió de disseny és intencional: centralitzar el *wiring* al controlador manté `MainFrame` com una classe de presentació pura sense cap coneixement de la lògica de negoci, i evita que la vista hagi de conèixer el controlador directament. El mètode registra set listeners: els tres botons d'execució (Iniciar, Aturar, Reiniciar), els tres de navegació (pas anterior, pas següent, reproduir/pausar), el lliscador de velocitat i el callback `onConfigChanged` de `ConfigPanel`. El listener del lliscador té un comportament notable: si l'animació ja està en curs, actualitza el retard del temporitzador en temps real (`animationTimer.setDelay()`) sense aturar ni reiniciar l'animació. El callback de configuració redimensiona el

tauler i refresca la previsualització de les peces cada vegada que l'usuari canvia les dimensions o el tipus de peça.

Animació pas a pas.. Quan l'usuari clica *Reproduir*, es crea un nou `javax.swing.Timer` amb el retard actual del lliscador; si la reproducció anterior havia arribat al final, el pas es reinicia a 0 automàticament. Cada *tick* del temporitzador crida `stepForward()`: si queden passos, incrementa `currentStep` i actualitza el tauler; si s'ha assolit l'últim pas, atura el temporitzador automàticament. En acabar la cerca, el mètode `onSolvingComplete()` gestiona tres casos: si el solver ha estat aturat, si ha trobat solució (en aquest cas carrega el camí i habilita la navegació) o si no existeix solució; en tots tres casos actualitza l'estat visual de la interfície de manera consistent.

Quan s'inicia la cerca:

- 1) Llegeix els paràmetres de `ConfigPanel` (dimensions, tipus i posicions de les peces blanca i negra).
- 2) Construeix un objecte `Board` i un `BacktrackingSolver`.
- 3) Crea un `SolverTask` i el posa en marxa amb `execute()`.
- 4) El callback de completació (`onSolvingComplete`) rep el resultat al fil EDT, carrega la solució amb `loadSolution()` i actualitza l'estat visual de la vista.

La Fig. 1 mostra el diagrama de classes simplificat amb les relacions entre les tres capes.

[Diagrama de classes MVC — vegeu suggeriment al codi font]

Figura 1. Diagrama de classes i relacions entre les tres capes MVC.

4. Anàlisi de Complexitat

4.1. Complexitat en el Pitjor Cas

Sigui $N \cdot M = S$ el nombre total de caselles. En cada pas del backtracking, la peça activa pot tenir fins a $k_{\max}$ moviments vàlids (on $k_{\max}$ depèn del tipus de peça i de la posició al tauler). L'arbre de cerca té profunditat màxima $S - 2$ (les dues caselles inicials ja estan marcades). Per tant, el cost en el pitjor cas és:

$$T_{\text{pitjor}}(S) = O(k_{\max}^{S-2}) \quad (1)$$

que és exponencial en la mida del tauler. Per a un tauler 8×8 (el tauler d'escacs estàndard, $S = 64$) i un cavall ($k_{\max} = 8$), el pitjor cas és $O(8^{62}) \approx 10^{55}$, completament inabordable sense heurística.

4.2. Heurística de Warnsdorff com a Optimització

L'heurística de Warnsdorff [1] és una regla *greedy* que ordena els moviments candidats creixentment pel nombre de moviments disponibles des de la casella destí cap a caselles no visitades. Formalment, donada la posició actual p , cada candidat q rep un pes:

$$w(q) = |\{r \in N(q) : r \notin V\}| \quad (2)$$

on $N(q)$ és el conjunt de caselles accessibles des de q i V el conjunt de caselles ja visitades. S'escull primer el candidat amb menor $w(q)$, ja que les caselles d'accés difícil convé visitar-les aviat per evitar quedar-se sense moviments.

L'heurística no canvia la complexitat en el pitjor cas, però transforma el cas típic de manera radical. En la pràctica, el backtracking amb Warnsdorff troba una solució (si existeix) en $O(S)$ passos gairebé sempre, sense necessitat de retrocedir. Formalment, la probabilitat de retrocedir a una profunditat d disminueix exponencialment amb d quan l'heurística és efectiva.

La Taula 3 compara el cost observat experimentalment per a diverses combinacions de peces i mides de tauler.

Taula 3. ITERACIONS I RETROCESOS OBSERVATS PER A DIVERSES CONFIGURACIONS (VALORS ORIENTATIUS).

Peces	Tauler	Iteracions	Retrocesos
Cavall + Cavall	8×8	~ 64	~ 0
Cavall + Torre	8×8	~ 100	~ 5
Cavall + Reina	6×6	~ 500	~ 20
Reina + Reina	6×6	$> 10^6$	$> 10^4$
Alfil + Mussol	5×5	~ 200	~ 10

Aquests valors mostren clarament l'efecte de l'heurística de Warnsdorff: per a peces amb moviments ben distribuïts (com el Cavall), l'heurística elimina gairebé tots els retrocesos, mentre que per a peces amb gran mobilitat i restriccions de captura severes (com dues Reines), el nombre de retrocesos creix exponencialment.

4.3. Cost de Comprovació de Captura

El mètode `canCapture(attackPos, attacker, targetPos)` té un cost proporcional al nombre de directions de la peça multiplicat per la mida màxima del tauler (per a peces contínues):

$$T_{\text{captura}} = O(d \cdot \max(N, M)) \quad (3)$$

on d és el nombre de vectors de la peça ($d \leq 8$ per a la Serp i el Mussol). Com que `canCapture` s'invoca dues vegades per a cada moviment candidat, el cost total de la comprovació de captura per iteració és $O(d \cdot \max(N, M))$.

4.4. Cost de l'Ordenació per Warnsdorff

Per a cada posició candidata q , calcular $w(q)$ requereix generar els moviments des de q i comptar-ne els no visitats:

cost $O(d \cdot \max(N, M))$. L'ordenació de k candidats és $O(k \log k)$. En total, el cost de l'ordenació per iteració és:

$$T_{\text{ord}} = O(k \cdot d \cdot \max(N, M) + k \log k) \quad (4)$$

5. Resultats i Discussió

5.1. Verificació de Correcció

Per a cada configuració resolta, l'aplicació verifica internament que:

- 1) Totes les caselles del tauler estan a la llista de posicions.
- 2) Cada moviment de la llista és vàlid per al tipus de peça corresponent.
- 3) Cap parell de posicions consecutives viola la restricció de no-captura.

Les proves manuals sobre taulers de mida 4×4 a 8×8 amb diverses combinacions de peces han confirmat la correcció de l'algorisme.

5.2. Influència de la Mida del Tauler

Com s'espera per un problema NP-hard, el temps de resolució creix molt ràpidament amb la mida del tauler quan la heurística no és suficient. Per a la combinació de dues reines, taulers de 7×7 o més ja presenten temps de resolució superiors a un minut.

Aquest comportament exponencial es deu a dos factors: primer, l'espai de cerca creix com $O(k^{N \cdot M})$, on k és el nombre mitjà de moviments per posició; segon, les restriccions de captura entre dues peces de gran abast (com reines) eliminen gairebé tots els candidats en profunditat mitjana-alta, forçant backtracking massiu. En contrast, taulers més grans amb peces de mobilitat limitada (com dos Cavalls) sovint es resolen més ràpidament perquè l'heurística de Warnsdorff té més opcions per escollir i pot evitar millor els camins sense sortida.

5.3. Influència del Tipus de Peça

El tipus de peça afecta tant la dificultat de la cerca com l'existència de solució. Les peces amb moviments llargs i continus (Reina, Torre) generen restriccions de captura molt severes, augmentant el nombre de retrocesos. Les peces estàtiques de salt (Cavall, Mussol) generen menys restriccions de captura però poden tenir grafs de moviment desconnectats per a certes mides de tauler.

Cal destacar una observació crítica: l'heurística de Warnsdorff, dissenyada originalment per al Cavall en un tauler estàndard, no és igualment efectiva per a totes les combinacions de peces. Per a peces contínues com la Reina, l'heurística pot produir resultats contraproduents: escollir la casella amb menys accessibilitat futura pot bloquejar ràpidament l'altra peça a causa de les restriccions de captura. Aquest fenomen explica per què certes combinacions (Reina + Reina) són exponencialment més costoses que d'altres, fins i tot en taulers petits.

5.4. Impacte de la Posició Inicial

La posició inicial de les peces influeix significativament en el temps de resolució. Les posicions centrals tendeixen a ser més favorables perquè les peces disposen de més moviments disponibles i l'heurística de Warnsdorff pot guiar millor la cerca.

No obstant això, existeixen casos excepcionals on posicions aparentment desfavorables (cantonades o marges) poden resultar en temps de resolució més curts. Aquest fenomen es deu al fet que començar en una posició restrictiva força l'algorisme a visitar zones difícils d'accés al principi del recorregut, evitant quedar atrapat en camins sense sortida més endavant. Aquest comportament suggereix que una estratègia híbrida —començant deliberadament per les zones més restrictives— podria millorar el rendiment mitjà de l'heurística.

5.5. Limitacions de l'Heurística

Tot i que l'heurística de Warnsdorff és altament efectiva per a moltes configuracions, presenta limitacions importants que han estat observades durant l'experimentació:

- 1) **Decisions greedy irrevocables:** L'heurística pren decisions locals òptimes que poden bloquejar globalment la solució. En particular, no considera l'impacte de la restricció de no-captura sobre l'altra peça.
- 2) **Empataments:** Quan múltiples caselles tenen el mateix grau de Warnsdorff, la tria arbitrària pot determinar l'èxit o fracàs de la cerca. Una exploració exhaustiva dels empataments milloraria la taxa d'èxit però incrementaria el cost.
- 3) **Peces amb gran mobilitat:** Per a peces com la Reina, el grau de Warnsdorff sovint és artificialment alt (gairebé totes les caselles són accessibles), fent que l'heurística perdi poder discriminatori.

Aquestes observacions suggereixen que una heurística adaptativa, que ajusti els seus criteris segons el tipus de peces i les restriccions actives, podria superar significativament la implementació estàndard de Warnsdorff.

5.6. Disseny del Programari

L'arquitectura MVC ha demostrat la seva utilitat durant el desenvolupament. Gràcies a la separació estricta de responsabilitats:

- L'algorisme de backtracking (`BacktrackingSolver`) s'ha pogut provar de manera aïllada, sense cap dependència de Swing.
- Afegir nous tipus de peces ha consistit únicament en afegir una constant a l'enum `PieceType` amb els seus vectors de moviment; cap altra classe ha necessitat modificació.
- La navegació pas a pas s'ha pogut implementar a la capa de la vista sense afectar el model ni el controlador.

6. Conclusions

Aquesta pràctica ha permès aplicar el paradigma del backtracking a un problema combinatori no trivial i comprovar experimentalment l'impacte de l'heurística de Warnsdorff sobre el rendiment de la cerca.

Des del punt de vista algorísmic, la principal conclusió és que el backtracking pur és impracticable per a taulers de mida moderada a causa de la seva complexitat exponencial, però que l'heurística de Warnsdorff el transforma en un mètode eficaç per a la majoria de configuracions, reduint el nombre de retrocesos de l'ordre dels milions a l'ordre de les desenes o centenars.

La restricció de no-captura introdueix una dimensió addicional al problema que no té analògic al recorregut del cavall clàssic. Combinacions de peces amb gran abast (Reina + Reina) generen situacions on la restricció elimina gairebé tots els moviments disponibles, fent la cerca molt costosa o fins i tot sense solució en taulers petits.

Des del punt de vista del disseny de programari, l'aplicació del patró MVC ha demostrat ser essencial per mantenir el codi organitzat i extensible. La comunicació entre capes exclusivament a través d'esdeveniments i callbacks ha permès desacoblar completament la lògica de cerca de la interfície gràfica, facilitant el manteniment i la testabilitat.

La gestió de la concurrència amb `SwingWorker` i l'aturada cooperativa amb indicadors `volatile` ha garantit una interfície sempre responsiva, fins i tot durant cerques de llarga durada.

Com a treball futur, es podria implementar la visualització del procés de cerca en temps real (mostrant la frontera de cerca activa), explorar heurístiques addicionals com el reconeixement de caselles aïllades, o implementar una cerca paral·lela per explorar múltiples branques simultàniament.

Referències

- [1] Conrad Barski, "Warnsdorff's rule for Knight's tours," Wikipedia, Jan. 2025. [Online]. Available: https://en.wikipedia.org/wiki/Knight%27s_tour#Warnsdorff's_rule
- [2] camickr, "How do I use SwingWorker in Java?," Stack Overflow, Apr. 2009. [Online]. Available: <https://stackoverflow.com/questions/782265/how-do-i-use-swingworker-in-java>
- [3] Oracle, "Worker Threads and SwingWorker," *The Java Tutorials: Concurrency in Swing*. [Online]. Available: <https://docs.oracle.com/javase/tutorial/uiswing/concurrency/worker.html>
- [4] Oracle, "Controlling Rendering Quality," *The Java Tutorials: 2D Graphics*. [Online]. Available: <https://docs.oracle.com/javase/tutorial/2d/advanced/quality.html>
- [5] Oracle, "RenderingHints (Java SE 8)," *Java Platform SE 8 API Specification*. [Online]. Available: <https://docs.oracle.com/javase/8/docs/api/java/awt/RenderingHints.html>
- [6] M. Dahmus and F. Kleinschmidt, "mouseClicked vs. mousePressed on Apple vs. Windows," *comp.lang.java.gui*, 2007. [Online]. Available: <https://comp.lang.java.gui.narkive.com/bAHvswQ2/mouseclicked-vs-mousepressed-on-apple-vs-windows>